Acoustic Speech Emotion Recognition

Using Machine Learning and Deep Learning

End-to-End Implementation Report

Project Type	Full-Stack AI / ML / DL Application
Datasets	RAVDESS (1,440 files) + TESS (2,800 files)
Emotions	Neutral · Calm · Happy · Sad · Angry · Fearful · Disgust · Surprised
Models	SVM · MLP · 1D CNN
Best Accuracy	CNN → 82.63% (Test)
Generated	November 04, 2026 16:18

This document provides a complete end-to-end description of the system architecture, datasets, feature engineering, model design, training results, API specification, and deployment instructions.

01 | Project Overview

Speech Emotion Recognition (SER) is the task of automatically identifying the emotional state of a speaker from raw audio. This project implements a complete, production-ready SER pipeline that trains three models (SVM, MLP, CNN), exposes them through a FastAPI REST service, and surfaces predictions through an interactive Next.js dashboard.

Objectives

- Classify speech recordings into 8 emotion categories.
- Compare classical ML (SVM, MLP) with deep learning (1D CNN).
- Provide real-time emotion prediction via a REST API.
- Visualise waveforms, spectrograms and confidence scores in a web UI.
- Persist prediction history with search, filter and delete.

System Architecture Flow

```
Audio Input (upload / microphone)
↓
Audio Preprocessing (resample → mono → trim → normalise → pre-emphasis → pad/cut to 3 s)
↓
Feature Extraction (MFCC-40 | Chroma-12 | Mel-128) → 180-dim vector | (130 × 180) matrix
↓ ↓ ↓
SVM / MLP Models CNN Model
↓
Softmax Probabilities → Predicted Emotion + Confidence
↓
FastAPI Response → Next.js Dashboard → SQLite History
```

02 | Technology Stack & Libraries

Backend (Python)

Library / Tool	Version	Purpose
Python	3.12.5	Core runtime
FastAPI	0.115.4	REST API framework
Uvicorn	0.32.0	ASGI server (programmatic)
Librosa	1.0.0	Audio loading, feature extraction
SoundFile	0.14.0	Audio I/O (WAV/FLAC/OGG)
NumPy	2.3.5	Numerical arrays
Pandas	2.2.3	Data manipulation
Scikit-learn	1.8.0	SVM, MLP, StandardScaler, metrics
TensorFlow / Keras	2.20.0	CNN model (Conv1D architecture)
Matplotlib	3.10.7	Training curves, confusion matrices
Seaborn	0.13.2	Heatmap confusion matrices
Joblib	1.4.2	Model serialisation (.pkl)
SQLite3	built-in	Prediction history persistence
python-multipart	0.0.12	Multipart form-data upload
protobuf	5.29.6	TensorFlow dependency (upgraded)
ReportLab	4.0.7	PDF report generation

Frontend (Node.js)

Library / Tool	Version	Purpose
Next.js	14.2.15	React SSR/SSG framework
React	18.3.1	UI component library
TypeScript	5.6.3	Static typing for JavaScript
Tailwind CSS	3.4.14	Utility-first CSS framework
Recharts	2.13.0	Chart.js-compatible React charts
Lucide React	0.453.0	SVG icon set
Axios / Fetch	built-in	HTTP client for API calls

Library / Tool	Version	Purpose
PostCSS	8.4.47	CSS transform pipeline

03 | Dataset Details

RAVDESS – Ryerson Audio-Visual Database of Emotional Speech and Song

- Total files used: 1,440 speech WAV files.
- Actors: 24 (12 male, 12 female), professional actors.
- Sampling rate: 48,000 Hz (resampled to 22,050 Hz during preprocessing).
- File naming convention: Modality-VocalChannel-Emotion-Intensity-Statement-Repetition-Actor.wav
- Emotion codes: 01=Neutral, 02=Calm, 03=Happy, 04=Sad, 05=Angry, 06=Fearful, 07=Disgust, 08=Surprised.
- Speaker-independent split: Actors 01, 09, 17, 19 held out for testing.

TESS – Toronto Emotional Speech Set

- Total files used: 2,800 WAV files.
- Speakers: 2 actresses (Young Adult & Older Adult).
- Emotions: 7 expressed (angry, disgust, fear, happy, neutral, ps, sad).
- Folder naming: YAF__/ and OAF__/.
- Emotion mapping: 'fear' → fearful, 'ps' → surprised (matches RAVDESS schema).
- Split: 20% stratified test set by emotion class.

Combined Dataset Summary

Dataset	Files	Emotions	Train	Test
RAVDESS	1,440	8	Speaker-independent actors	4 actors held out
TESS	2,800	7 (mapped to 8)	80% stratified	20% stratified
Total	**4,240**	**8**	**~3,440**	**~800**

■ **Note on RAVDESS:** TESS does not contain 'calm' samples; the calm class is populated exclusively from RAVDESS. Class imbalance is managed through stratified splitting and weighted evaluation metrics.

04 | Audio Preprocessing Pipeline

All audio is processed through an identical pipeline at both training time and inference time, ensuring no train/test distribution mismatch.

Step	Operation	Configuration
1	Load Audio	librosa.load() with native sample rate
2	Resample	Target SR = 22,050 Hz
3	Stereo → Mono	Channel averaging (librosa mono=True)
4	Silence Trimming	librosa.effects.trim(top_db=20)
5	Amplitude Normalisation	Scale to peak 0.95 (prevent clipping)
6	Pre-emphasis Filter	coefficient = 0.97 (boost high frequencies)
7	Fixed-Length Padding/Cutting	Duration = 3.0 s → 66,150 samples

Key Configuration Constants

```
SAMPLE_RATE = 22,050 Hz
DURATION = 3.0 seconds
N_SAMPLES = SAMPLE_RATE × DURATION = 66,150
N_FFT = 2,048
HOP_LENGTH = 512
N_MFCC = 40
N_CHROMA = 12
N_MELS = 128
```

05 | Feature Extraction

Two feature representations are extracted from every audio clip — one for the ML models (tabular) and one for the CNN (sequential).

Traditional ML Feature Vector (180 dimensions)

Feature Group	Algorithm	Dims	Description
MFCC	librosa.feature.mfcc	40	Mean of 40 Mel-Frequency Cepstral Coefficients across time
Chroma	librosa.feature.chroma_stft	12	Mean of 12 Chroma pitch-class energy bins
Mel Spectrogram	librosa.feature.melspectrogram	128	Mean of 128 Mel-band energies (log scale)
Total	Concatenated	**180**	Single flat feature vector per recording

CNN Sequential Feature Matrix (130 × 180)

Rather than averaging across time, the CNN receives the full time-frame sequence:

- Each column in the matrix corresponds to one of the 180 feature dimensions.
- Each row corresponds to one time frame (hop_length=512 steps along the audio).
- With 66,150 samples and hop_length=512 → approximately 130 time frames.
- Shape fed to Conv1D: (batch, 130, 180) — time steps as sequence length, features as channels.

Feature Cache

All 4,240 feature matrices are computed once and cached to **backend/models/features_cache.npz** (≈ 227 MB). Subsequent training runs skip recomputation, reducing setup time from ~20 min to under 10 s.

06 | Machine Learning Models

Model 1 – Support Vector Machine (SVM)

File: **backend/training/train_svm.py** | Saved: **backend/models/svm_model.pkl**

Component	Configuration
Scaler	StandardScaler() — zero mean, unit variance
Classifier	SVC(kernel='rbf', C=10, gamma='scale', probability=True)
Pipeline	sklearn.pipeline.Pipeline([scaler, svc])
Input	180-dimensional feature vector
Output	8-class probability vector via Platt scaling
Serialisation	joblib.dump() → svm_model.pkl

Model 2 – Multi-Layer Perceptron (MLP)

File: **backend/training/train_mlp.py** | Saved: **backend/models/mlp_model.pkl**

Hyperparameter	Value	Rationale
hidden_layer_sizes	(300,)	Single hidden layer, 300 neurons — avoids overfitting on tabular features
alpha	0.01	L2 regularisation strength
batch_size	256	Mini-batch gradient descent
learning_rate	'adaptive'	Halves LR when training loss stops improving
max_iter	500	Maximum training epochs
activation	'relu'	Default; non-linear hidden units
solver	'adam'	Default; works well with large datasets
Scaler	StandardScaler()	Same scaler as SVM, fit on training data only
Serialisation	joblib.dump()	→ mlp_model.pkl

07 | Deep Learning Model – 1D CNN

File: **backend/training/train_cnn.py** | Saved: **backend/models/cnn_model.keras**

Layer	Type	Parameters	Output Shape
1	Input	shape=(130, 180)	(None, 130, 180)
2	Conv1D	filters=64, kernel_size=5, padding='same'	(None, 130, 64)
3	ReLU	—	(None, 130, 64)
4	Dropout	rate=0.25	(None, 130, 64)
5	MaxPooling1D	pool_size=2	(None, 65, 64)
6	Conv1D	filters=128, kernel_size=5, padding='same'	(None, 65, 128)
7	ReLU	—	(None, 65, 128)
8	Dropout	rate=0.30	(None, 65, 128)
9	MaxPooling1D	pool_size=2	(None, 32, 128)
10	Conv1D	filters=256, kernel_size=3, padding='same'	(None, 32, 256)
11	ReLU	—	(None, 32, 256)
12	Dropout	rate=0.30	(None, 32, 256)
13	Flatten	—	(None, 8,192)
14	Dense	units=8, activation='softmax'	(None, 8)

Training Configuration

Setting	Value
Loss function	Categorical Cross-Entropy
Optimiser	Adam (learning_rate=0.001, beta_1=0.9, beta_2=0.999)
Batch size	32
Max epochs	100
Validation split	15% of training data
EarlyStopping	patience=15, monitor='val_accuracy', restore_best_weights=True
ModelCheckpoint	save_best_only=True → cnn_model.keras
Feature normalisation	Per-channel StandardScaler → cnn_scaler.npz
Total parameters	328,392

08 | Model Evaluation Results

All metrics are computed on the held-out test set (~800 samples). No metric is hardcoded — values reflect actual empirical training results.

Overall Performance Comparison

Model	Train Acc	Test Acc	Precision	Recall	F1-Score
SVM	99.45%	79.12%	81.54%	79.12%	79.68%
MLP	99.19%	79.75%	81.15%	79.75%	79.77%
CNN	93.26%	82.63%	84.34%	82.62%	82.30%

Key Observations

- CNN outperforms both ML models on test accuracy (+2.88% over MLP, +3.51% over SVM).
- SVM and MLP show significant train/test gaps (overfitting), while CNN generalises better with dropout.
- CNN benefits from modelling temporal dynamics in the (130×180) feature matrix.
- Precision slightly exceeds Recall for all models, indicating fewer false positives than false negatives.
- Best-performing model: CNN with 82.63% test accuracy.

Generated Evaluation Artefacts

File	Description
models/svm_confusion_matrix.png	8×8 heatmap – SVM test predictions vs ground truth
models/mlp_confusion_matrix.png	8×8 heatmap – MLP test predictions vs ground truth
models/cnn_confusion_matrix.png	8×8 heatmap – CNN test predictions vs ground truth
models/cnn_training_curves.png	4-subplot figure: train/val accuracy & train/val loss vs epoch
models/model_comparison.png	Grouped bar chart comparing accuracy, precision, recall, F1
models/evaluation_metrics.json	Full metrics dict (accuracy, precision, recall, f1, per-class report)

Train/Test Split Strategy

- RAVDESS: Speaker-independent split — Actors 01, 09, 17, 19 (4 of 24) form the test set. No utterance from a test actor appears in training data.
- TESS: 20% stratified random split by emotion label.
- Data leakage prevention: Scalers are fit exclusively on training data; test data is never seen during preprocessing.

09 | Backend API Specification

The FastAPI server starts with **python backend/main.py**. Uvicorn is launched programmatically — no separate uvicorn command is required. CORS is enabled for **http://localhost:3000**.

Method	Endpoint	Purpose	Request	Response
GET	/	Backend status check	—	200 JSON status
GET	/health	Health + model load status	—	200 JSON health
GET	/models	List loaded models + accuracies	—	200 JSON model list
GET	/analytics	Full evaluation metrics	—	200 JSON metrics
POST	/predict	Predict emotion from audio	multipart/form-data: file, model	200 JSON prediction
POST	/predict-all	Run all 3 models, compare	multipart/form-data: file	200 JSON all predictions
POST	/recording	Accept microphone blob	multipart/form-data: audio blob	200 JSON prediction
GET	/history	Retrieve prediction history	Query params: limit, offset, emotion, model	200 JSON history list
DELETE	/history/{id}	Delete one history record	Path param: id	200 JSON confirmation

Prediction Response Schema

```
{
"predicted_emotion": "happy",
"confidence": 89.5,
"model": "CNN",
"probabilities": {
"happy": 89.5, "sad": 2.5, "angry": 3.0, "neutral": 2.0,
"fearful": 1.0, "disgust": 0.5, "surprised": 1.0, "calm": 0.5
},
"filename": "test.wav",
"processing_time_ms": 142
}
```

10 | Frontend Pages & Components

The Next.js frontend is a 4-page application with an App Router structure. All API calls are centralised in **frontend/lib/api.ts**.

Dashboard (app/dashboard/page.tsx)

Overview metrics (total recordings, predictions, models, best accuracy). Emotion distribution bar chart (Recharts). Model accuracy comparison. Recent predictions table. Live backend health badge (via Navbar).

- Key components: Navbar, Sidebar, EmotionCard, ConfidenceChart, ModelComparison

Emotion Prediction (app/predict/page.tsx)

Drag-and-drop audio upload with file validation. Microphone recording with live timer. Audio playback (element). Model selector (SVM / MLP / CNN / All). Predict button → shows emotion, confidence gauge, probability chart, waveform visualisation.

- Key components: AudioUploader, AudioRecorder, EmotionCard, ConfidenceChart, Waveform, ModelComparison

Model Analytics (app/analytics/page.tsx)

Per-model accuracy, precision, recall, F1-score cards. Accuracy comparison bar chart. CNN training/validation accuracy and loss curves (Recharts LineChart). Per-class F1 heatmaps. Confusion matrix displays.

- Key components: ModelComparison, ConfidenceChart

Prediction History (app/history/page.tsx)

Paginated table: filename, emotion, confidence, model, timestamp. Search by filename. Filter by emotion or model. Delete individual records (DELETE /history/{id}). History persisted in SQLite via backend/services/history_service.py.

- Key components: Sidebar, Navbar

Shared Components

Component	File	Description
Navbar	components/Navbar.tsx	Top bar with live API health indicator
Sidebar	components/Sidebar.tsx	Left nav drawer with model accuracy summary
AudioUploader	components/AudioUploader.tsx	Drag-and-drop + browse, audio playback
AudioRecorder	components/AudioRecorder.tsx	MediaRecorder API, live waveform, stop/save
EmotionCard	components/EmotionCard.tsx	Emoji badge, emotion name, confidence ring
ConfidenceChart	components/ConfidenceChart.tsx	Recharts horizontal bar chart of probabilities
Waveform	components/Waveform.tsx	Canvas-based acoustic waveform visualiser

Component	File	Description
ModelComparison	components/ModelComparison.tsx	3-column side-by-side model result cards

11 | Project Folder Structure

```
Acoustic-Speech-Emotion-Recognition/
■■■ frontend/
■ ■■■ app/
■ ■ ■■■ layout.tsx # Root layout (Navbar + Sidebar)
■ ■ ■■■ page.tsx # Redirect → /dashboard
■ ■ ■■■ globals.css
■ ■ ■■■ dashboard/page.tsx # Overview metrics & charts
■ ■ ■■■ predict/page.tsx # Upload / record / predict
■ ■ ■■■ analytics/page.tsx # Model performance deep-dive
■ ■ ■■■ history/page.tsx # SQLite history table
■ ■■■ components/
■ ■ ■■■ Navbar.tsx AudioUploader.tsx
■ ■ ■■■ Sidebar.tsx AudioRecorder.tsx
■ ■ ■■■ EmotionCard.tsx ConfidenceChart.tsx
■ ■ ■■■ Waveform.tsx ModelComparison.tsx
■ ■■■ lib/api.ts # Centralised API client
■ ■■■ package.json tsconfig.json next.config.js
■ ■■■ tailwind.config.js postcss.config.js
■ ■■■ .env.local # NEXT_PUBLIC_API_URL=http://localhost:8000
■
■■■ backend/
■ ■■■ datasets/
■ ■ ■■■ RAVDESS/ # 1,440 WAV files
■ ■ ■■■ TESS/ # 2,800 WAV files
■ ■■■ models/
■ ■ ■■■ svm_model.pkl # Trained SVM pipeline
■ ■ ■■■ mlp_model.pkl # Trained MLP pipeline
■ ■ ■■■ cnn_model.keras # Best CNN checkpoint
■ ■ ■■■ label_encoder.pkl # 8-class LabelEncoder
■ ■ ■■■ cnn_scaler.npz # CNN feature normalisation
■ ■ ■■■ features_cache.npz # Precomputed features (~227 MB)
■ ■ ■■■ evaluation_metrics.json # Actual training results
■ ■■■ training/
■ ■ ■■■ dataset_loader.py feature_extraction.py
■ ■ ■■■ preprocessing.py train_svm.py
■ ■ ■■■ train_mlp.py train_cnn.py
■ ■ ■■■ evaluate_models.py visualize_results.py
■ ■ ■■■ download_datasets.py
■ ■■■ services/
■ ■ ■■■ prediction_service.py # SVM/MLP/CNN inference
■ ■ ■■■ audio_service.py # Validation, waveform
■ ■ ■■■ history_service.py # SQLite CRUD
■ ■■■ uploads/ # Temporary uploaded files
■ ■■■ main.py # FastAPI app + Uvicorn
■ ■■■ config.py # Central configuration
```

```
■ ■■■ requirements.txt
■
■■■ README.md
```

12 | Execution Instructions

Prerequisites

- Python 3.10+ (tested on 3.12.5)
- Node.js 18+ and npm 9+ (tested on Node 24.11.1 / npm 11.6.2)
- RAVDESS and TESS datasets in backend/datasets/ (or run download_datasets.py)

Step 1 – Install Backend Dependencies

```
cd backend
pip install -r requirements.txt
```

Step 2 – Train Models (one-time)

Run each training script from the backend/ directory. Features are cached automatically.

```
python training/train_svm.py
python training/train_mlp.py
python training/train_cnn.py
python training/evaluate_models.py # (optional – regenerates metrics JSON)
```

Step 3 – Start Backend Server

```
cd
python backend/main.py
# → Uvicorn starts on http://localhost:8000
```

Step 4 – Install Frontend Dependencies

```
cd frontend
npm install
```

Step 5 – Start Frontend Dev Server

```
npm run dev
# → Next.js starts on http://localhost:3000
```

Step 6 – Open the Application

Navigate to **http://localhost:3000** in any modern browser. The Navbar will show a green 'API Connected' badge when the backend is reachable.

requirements.txt (Backend)

```
fastapi==0.115.4
uvicorn[standard]==0.32.0
python-multipart==0.0.12
librosa==1.0.0
soundfile==0.14.0
numpy==2.3.5
pandas==2.2.3
```

```
scikit-learn==1.8.0
tensorflow==2.20.0
matplotlib==3.10.7
seaborn==0.13.2
joblib==1.4.2
protobuf==5.29.6
reportlab==4.0.7
```

13 | Ethical Considerations & Limitations

Disclaimer

Acoustic emotion recognition is a research-grade technology with important limitations. Users and deployers of this system must be aware of the following:

- **Estimates, not diagnoses:** Emotion predictions are probabilistic estimates based on acoustic features. They do not constitute psychological assessments and must never be used as clinical or legal evidence.
- **Cultural variability:** Emotional expression varies significantly across cultures, languages, and individuals. Models trained on North American English datasets (RAVDESS, TESS) may generalise poorly to other populations.
- **Acted vs. spontaneous speech:** Both datasets use professionally acted speech, which may differ substantially from naturalistic emotional expression in real-world conditions.
- **Speaker characteristics:** Accuracy may vary based on age, gender, accent, vocal quality, and speaking style not represented in the training data.
- **Consent and privacy:** Do not record or analyse individuals' speech without explicit informed consent. Audio data should be handled according to applicable privacy regulations (GDPR, CCPA, etc.).
- **Bias:** The 8-class taxonomy may not capture the full nuance of human emotional experience. Class imbalance (calm is RAVDESS-only) may affect per-class performance.
- **Responsible deployment:** This system is intended for research, educational, and demonstration purposes. Any production deployment in sensitive domains (healthcare, law enforcement, HR) requires additional validation, bias auditing, and regulatory review.

All confidence scores represent the model's internal probability estimates, not guarantees of correctness. The predicted emotion should always be interpreted in context and with human oversight.

14 | References & Acknowledgements

RAVDESS Dataset: Livingstone SR, Russo FA (2018). The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS). PLoS ONE 13(5): e0196391. https://doi.org/10.1371/journal.pone.0196391

TESS Dataset: Dupuis K, Pichora-Fuller MK (2010). Toronto Emotional Speech Set (TESS). University of Toronto Psychology, Rotman Research Institute.

Librosa: McFee B et al. (2015). librosa: Audio and Music Signal Analysis in Python. Proceedings of the 14th Python in Science Conference.

FastAPI: Ramírez S. FastAPI framework. https://fastapi.tiangolo.com

TensorFlow / Keras: Abadi M et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. OSDI 2016.

Scikit-learn: Pedregosa F et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825–2830.

Next.js: Vercel. Next.js – The React Framework. https://nextjs.org

---

Generated: November 04, 2026 at 16:18

Acoustic Speech Emotion Recognition | ML & DL Implementation Report